

TRADING AGENT

Strategy C — Agentic Architecture

7 agents · Haiku + Sonnet · Scanner Agent · Learning Agent · Full trace observability

Phase 0 — Simulation · June 2026

Why Strategy C

Strategy A / B (pipeline)

- Python pre-packages all context
- One Claude call receives it all
- Claude outputs a list and exits
- No tool access — no agency
- Static config, human-updated only
- Minimal trace logging

Strategy C (agentic)

- Agents decide what to look up via tools
- 7 specialized agents, each minimal
- Scanner Agent: regime-aware, dynamic N
- Orchestrator coordinates the loop
- Research Agent runs per ticker in parallel
- Learning Agent adjusts params nightly
- Full trace log to `c_traces` from day one

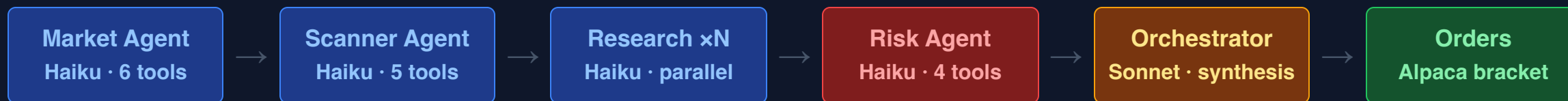
The inversion: Claude agents control information gathering instead of receiving a pre-built package.

Seven-Agent Roster

Agent	Model	Language	Role	Tools
Market Agent	Haiku	Python	GO / CAUTION / SKIP + session params	6
Scanner Agent	Haiku	Python	Regime-aware ticker selection, dynamic N	5
News Analyst	Haiku	TypeScript	Per-candidate news (Phase 0: stubbed)	2
Research Agent	Haiku	Python xN	Per-ticker conviction score, parallel	4
Risk Agent	Haiku	Python	Validates each trade proposal individually	4
Orchestrator	Sonnet	Python	Coordinates the loop, final trade decisions	5
Learning Agent	Sonnet	Python	EOD reflection, adjusts strategy params	7

Haiku for cost-sensitive data retrieval · Sonnet for reasoning, synthesis, and learning

Premarket Loop



Scanner Agent receives `session_params` from Market Agent and selects tickers based on regime:

- High VIX (>25) → mean-reversion signals, smaller N
- Low VIX (<15) → momentum signals, larger N
- CAUTION → strict threshold, $N \leq 15$. Dynamic N: 12–30 tickers per day

Retry gate — triggered when Risk Agent rejects proposals:

- Re-synthesize with rejection reasons in context (`loop_iteration = 2`)
- All rejected → re-run Research Agent first, then re-synthesize
- CAUTION mode: retry disabled regardless

Hard limit: max 2 loop iterations. Output: `{ trades[], retry_needed, terminal_reason }`

Market Agent — Six Tools

Market Agent

claude-haiku-4-5

Pure data retrieval — Haiku chosen for cost.
No reasoning needed, just structured tool calls.

Output: `session_params`

→ decision: **GO** / **CAUTION** / **SKIP**

→ `max_positions` , `vix` , `fear_greed` , `sector_bias`

Tool	Source
<code>get_vix()</code>	yfinance <code>^VIX</code>
<code>get_futures()</code>	yfinance ES=F, NQ=F, YM=F
<code>get_fear_greed()</code>	alternative.me (0–100)
<code>get_sector_etfs()</code>	Alpaca: XLK, XLE, XLF...
<code>get_economic_calendar()</code>	yfinance key events
<code>set_session_params()</code>	writes decision to context

Research Agent — Parallel Per Ticker

Research Agent

`claude-haiku-4-5`

One Haiku instance per candidate,
all running concurrently via asyncio.

Hard token budget enforced per instance.

Output per ticker:

- `research_summary` (text)
- `conviction_score` (0–10)

All summaries fed to Orchestrator.

4 tools per instance:

Tool	Fetches
<code>get_price_history()</code>	RSI, MACD, SMA50/200, ATR
<code>get_technical_indicators()</code>	Computed, normalized scores
<code>get_sector_context()</code>	ETF vs ticker performance
<code>get_peer_comparison()</code>	Ticker vs its closest peers

Each agent is independent — no shared state between parallel instances.

Orchestrator + Retry

Orchestrator

`claude-sonnet-4-6`

Reads all research summaries and market context. Proposes a trade list with entry, target, stop, confidence, and rationale.

Loop iteration 1 → Risk Agent.

If retry needed → iteration 2.

Iteration 2 is terminal.

5 tools:

Tool	Purpose
<code>load_params()</code>	Read <code>c_strategy_params</code>
<code>get_pending_trades()</code>	Avoid double-entry
<code>approve_trade()</code>	Add to pending queue
<code>reject_trade()</code>	Log rejection + reason
<code>write_session_meta()</code>	Write cost + outcome

CAUTION mode skips the retry loop entirely, regardless of `retry_needed` flag.

Protection System — Six Tiers

`check_protection_status()` runs at the start of every session. Events in `c_protection_events` are immutable.

Tier	Trigger	Action	Duration
1	(reserved)	Log only	—
2	<code>daily_pnl ≤ -\$500</code>	Suspend remainder of today	End of day
3	<code>rolling_3d_pnl ≤ -\$1,000</code>	Reduce <code>max_positions</code> by 50%	2 days
4	Account drawdown ≥ 10%	Suspend all trading	24 hours
5	Account drawdown ≥ 20%	Suspend — human unlock required	7 days
6	5+ consecutive losing days	Suspend — human review required	7 days

EOD — Learning Agent

Sequence: Force close → Reconcile fills → Compute performance → Check protection → **Learning Agent**

4 Read tools

- `read_today_trades()`
- `read_session_context()`
- `read_strategy_params()`
- `read_recent_learnings()`

3 Write tools

- `write_learning()` → `c_learnings`
- `adjust_param()` → `c_strategy_params`
- `recommend_goal()` → human approval req

Guards: min 3 trades to run · max 2 param changes/day · cooldown per param · bounds per param · safety params blocked (`daily_loss_limit` , `enable_learning_agent`)

Updated `c_strategy_params` feed into tomorrow's `load_params()` .

Trace Observability — Three Formats, One Table

Python JSON Traces

`TraceLogger` — all Python agents

- `log_tool_call()`
- `log_agent_message()`
- `log_decision()`
- `log_error()`

Session → agent → tool
span hierarchy

TypeScript OTel Spans

News Analyst

- `OTEL_SPAN: {json}` log lines

Fields: `agent.name` ,
`session.id` , `model`
`normalize_otel_span()`
ingests into `c_traces`

Structured Log Lines

Learning Agent

- `{ISO} [learning_agent]`
- `session={sid} event=...`
- `normalize_log_line()`

ingests into `c_traces`

`c_traces` — single normalized table: `session_id · span_id · parent_span_id · agent · tool_name · tool_input · tool_output · outcome · tokens_in · tokens_out · latency_ms · cost_usd`

Data Model — 12 Tables, Three Domains

Execution

- `c_sessions` — cost rollup, queue
- `c_positions` — fills, P&L, trail
- `c_scan_results` — scanner output
- `c_agent_config` — static config
- `c_market_evals` — shadow compare

Adaptive

- `c_strategy_params` — tunable params, cooldowns, bounds
- `c_learnings` — observations, adjustments, avoid_ticker
- `c_goals` — daily target + floor
- `c_goal_snapshots` — daily history

Observability

- `c_traces` — every tool call + cost
- `c_protection_events` — immutable
- `c_daily_performance` — P&L, win rate

Cost Model + Deployment Phases

Cost per session

Component	Est. cost
Market Agent (Haiku)	~\$0.001
Research x10 (Haiku)	~\$0.020
Orchestrator (Sonnet)	~\$0.050
Learning Agent (Sonnet)	~\$0.030
Total	~\$0.10 – 0.30

Strategy A/B for comparison: ~\$0.01 – 0.05

Deployment phases

Phase 0 — Simulation. No real orders. Traces on.

Phase 1 — Shadow alongside Strategy A. Compare outcomes.

Phase 2 — Paper capital. 2-week validation gate.

Phase 3 — Real capital after gate + 4-week paper run.

Not in Strategy A or B:

Tool use · Parallel per-ticker research

Retry loop · Adaptive params nightly

Full trace log from day one